

we need to set up serial and baud rate for Bluetooth as shown in Fig. 3. Set up a pin for servo using `servo.attach` (PWM pin number) function. Then define pin mode as output for LEDs.

In the third part of the code, create a loop, and check the incoming data from Bluetooth. Next, create 'if statement' to check the device ID. If it matches, create OTP () function for generation of OTP as shown in Fig. 4.

In the last part of the code, as shown in Fig. 5, create a `check ( )` function to check whether OTP is correct or not. If it is correct, turn the servo to an open position to unlock the locking system.

Congrats, you have done Arduino coding part!

Now, let's develop an Android app for our project. We can develop an Android app on two different platforms—Android Studio and MIT App Inventor. Here, let's choose the MIT App Inventor because it's easy to make an app with blocks and without normal coding.

Android app development

First, you need to have a valid Google email account. Next, open the link <https://appinventor.mit.edu> to start the MIT App Inventor project. You will find two tabs—Designer and Blocks on the top right side of the screen. Create a layout from the designer section, as shown in Fig. 6. Just drag and drop the components from the

Connections with Arduino Uno and Other Components

Arduino pins	Components and pin
Pin 9 (PWM)	Servo yellow wire (signal input wire)
Rx	Bluetooth module Tx
Tx	Bluetooth module Rx
GND	Bluetooth (-Ve)
Vcc	5V battery (+ Ve)
Pin 12	Red LED (Anode)

user interface (UI) on the left side or download the .aia file from source.efymag.com.

Now, select the Blocks tab to start blocks coding. (In this platform, you can program the app's behaviour by putting blocks together). First, we need to initialise Bluetooth list available for connection from the App Inventor Blocks Editor section. Then we need to set a button function to send device ID according to our Arduino code, as shown in Fig. 7.

Now, save the file as .apk on your computer. Transfer .apk file to your Android smartphone and install it.

We are almost done now. We have completed Arduino coding and Android app development. Now, let's start our hardware setup and connections. The circuit connections with the Arduino Uno board and other components are given in the table and also shown in Fig. 8.

Next, let's crosscheck everything, including connections. Wrong connections can damage your costly Arduino board. Do not connect servo motor directly to the 5V pin of Arduino.

It is not safe because Arduino has not been made for the amount of current that is required for servo motor to operate. Use a separate 5V DC supply for the servo motor. For interfacing the servo motor with the actual lock, you may refer to the 'Make Your Own Smart Wireless Biometric Lock' DIY article published in February issue.

Congrats, you have completed your awesome project! Now, switch on your OTP

```
void otp() {
    otp = letters[random(0, 6)] + numbers[random(0, 4)];
    Serial.println(otp + "\n");
}

void check() {
    while (Serial.available() == 0);
    pwd = Serial.readStringUntil('\n');

    if (pwd == otp) {
        Serial.println("unlocked");
        servo.write(120);
        digitalWrite(sled2, 1);
        digitalWrite(sled1, 0);
    }
    if (pwd != otp) {
        Serial.println("reset try again");
        servo.write(50);
        digitalWrite(sled2, 0);
    }
}
```

Fig. 5: Check () function

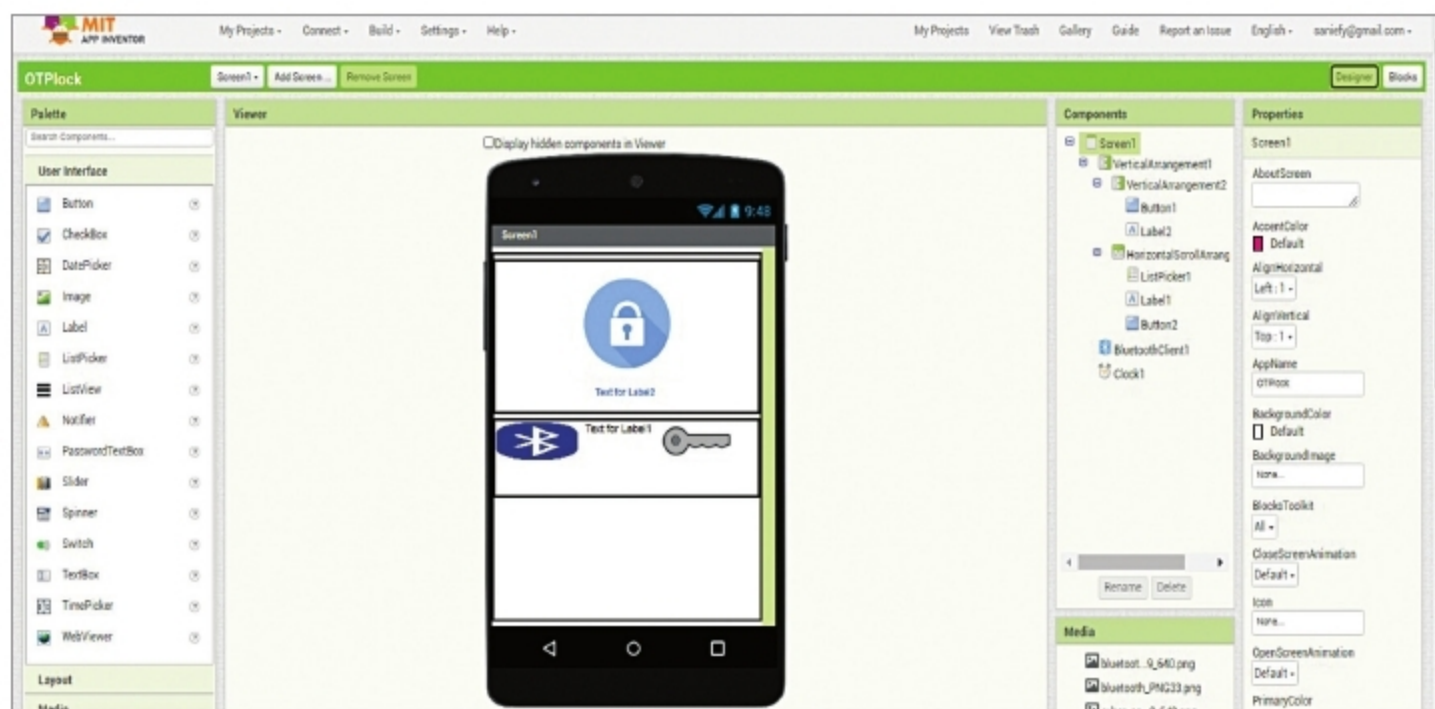


Fig. 6: Smart lock layout for Android app